École Nationale des Sciences Appliquées
Béni Mellal

Compte Rendu

TP N°4 : Architecture Orientée Services (SOA)

Encadrant : Pr. BE ELBAGHAZAOUI
Réalisé par : NAWAR EL ASRAOUI

0 Table des matières

1 Introduction **2**
1.1 Objectifs pédagogiques 2
1.2 Outils utilisés 2

2 Architecture du Système **2**

3 Étape 1 : Eureka Server **2**
3.1 Configuration `application.properties` 2
3.2 Classe principale 3

4 Étape 2 : Product Service **3**
4.1 Configuration 3
4.2 Modèle `Product.java` 3
4.3 Repository 4
4.4 Service et Contrôleur REST 4

5 Étape 3 : Order Service **5**
5.1 Configuration 5
5.2 Modèle `Order.java` 5
5.3 Client Feign 5
5.4 Logique métier – `OrderService.java` 5
5.5 Classe principale 6

6 Tests avec Postman **6**
6.1 Test 1 : Créer un produit 6
6.2 Test 2 : Lister les produits 6
6.3 Test 3 : Créer une commande 7
6.4 Test 4 : Lister les commandes 8

7 Résultats et Analyse **9**
7.1 Résumé des tests 9
7.2 Points clés observés 9

8 Conclusion **9**

1 Introduction

L'Architecture Orientée Services (SOA) est un paradigme de conception logicielle qui structure une application en services indépendants, réutilisables et communicants. Ce TP vise à implémenter une mini-application e-commerce en microservices avec Spring Boot et Spring Cloud.

1.1 Objectifs pédagogiques

- Comprendre les principes de l'architecture SOA
- Créer des microservices avec Spring Boot
- Implémenter la communication inter-services via REST et OpenFeign
- Gérer une base de données avec Spring Data JPA (H2)
- Utiliser un service de découverte (Eureka)

1.2 Outils utilisés

Outil	Version
Java JDK	17
Spring Boot	3.5.13
Spring Cloud	2024.x
Maven / mvnd	3.9+
IDE	IntelliJ IDEA
Test API	Postman

2 Architecture du Système

L'application e-commerce est composée de trois microservices principaux :

Service	Port	Rôle
Eureka Server	8761	Registre et découverte des services
Product Service	8081	Gestion du catalogue de produits
Order Service	8082	Gestion des commandes clients

Chaque service dispose de sa propre base de données H2 en mémoire, garantissant l'indépendance des services. La communication entre Order Service et Product Service est assurée par **OpenFeign** via le registre Eureka.

3 Étape 1 : Eureka Server

3.1 Configuration `application.properties`

```
server.port=8761
spring.application.name=eureka-server
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Listing 1 – application.properties – Eureka Server

3.2 Classe principale

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Listing 2 – EurekaServerApplication.java

4 Étape 2 : Product Service

4.1 Configuration

```
server.port=8081
spring.application.name=product-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:productdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

Listing 3 – application.properties – Product Service

4.2 Modèle `Product.java`

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

Listing 4 – Product.java

4.3 Repository

```java
@Repository
public interface ProductRepository extends JpaRepository<Product,
    Long> {
}
```

Listing 5 – ProductRepository.java

4.4 Service et Contrôleur REST

Le `ProductService` expose les opérations CRUD via `ProductController` sur le chemin `/api/products`. Les endpoints disponibles sont :

- `GET /api/products` – Lister tous les produits
- `GET /api/products/{id}` – Récupérer un produit par ID
- `POST /api/products` – Créer un produit
- `DELETE /api/products/{id}` – Supprimer un produit

```java
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;

    @GetMapping
    public List<Product> getAllProducts() {
        return productService.getAllProducts();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable
        Long id) {
        return productService.getProductById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        return productService.createProduct(product);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id
        ) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

Listing 6 – ProductController.java

5 Étape 3 : Order Service

5.1 Configuration

```
server.port=8082
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:orderdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

Listing 7 – application.properties – Order Service

5.2 Modèle `Order.java`

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long productId;
    private String productName;
    private Integer quantity;
    private Double totalPrice;
    private LocalDateTime orderDate;
    private String status; // PENDING, CONFIRMED, SHIPPED,
       DELIVERED
}
```

Listing 8 – Order.java

5.3 Client Feign

```
@FeignClient(name = "product-service")
public interface ProductClient {
    @GetMapping("/api/products/{id}")
    ProductDTO getProductById(@PathVariable Long id);
}
```

Listing 9 – ProductClient.java

5.4 Logique métier – `OrderService.java`

```java
public Order createOrder(Long productId, Integer quantity) {
    ProductDTO product = productClient.getProductById(productId);
    Order order = new Order();
    order.setProductId(productId);
    order.setProductName(product.getName());
    order.setQuantity(quantity);
    order.setTotalPrice(product.getPrice() * quantity);
    order.setOrderDate(LocalDateTime.now());
    order.setStatus("PENDING");
    return orderRepository.save(order);
}
```

Listing 10 – OrderService.java

5.5 Classe principale

```java
@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

Listing 11 – OrderServiceApplication.java

6 Tests avec Postman

6.1 Test 1 : Créer un produit

Requête : `POST http://localhost:8081/api/products`

```json
{
    "name": "Laptop",
    "description": "Ordinateur portable haute performance",
    "price": 1200.00,
    "stock": 10
}
```

Listing 12 – Body JSON – Création produit

6.2 Test 2 : Lister les produits

Requête : `GET http://localhost:8081/api/products`

Résultat : le produit **Laptop** est retourné avec `Status: 200 OK`.

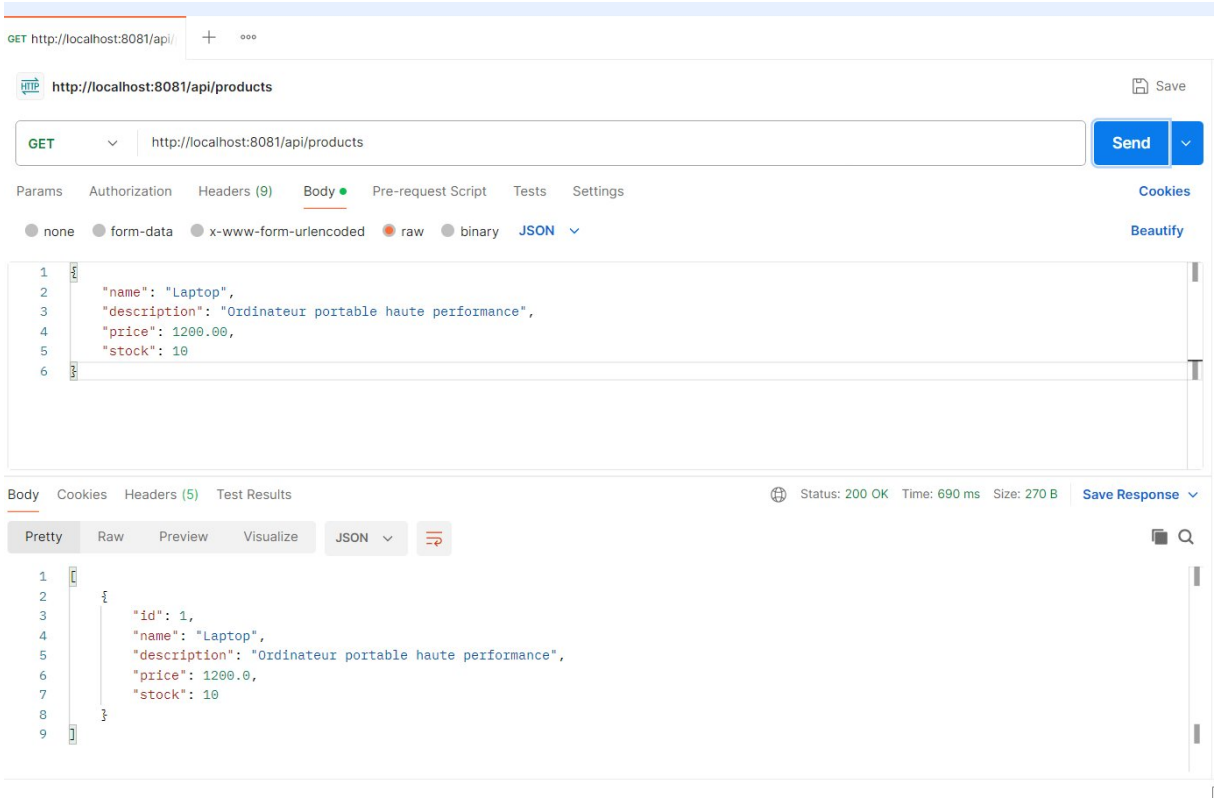

FIGURE 1 – GET /api/products – Liste des produits (200 OK)

6.3 Test 3 : Créer une commande

Requête : `POST http://localhost:8082/api/orders?productId=1&quantity=2`

Le Order Service appelle Product Service via Feign pour récupérer le prix, puis calcule automatiquement `totalPrice = 1200.0 x 2 = 2400.0`.

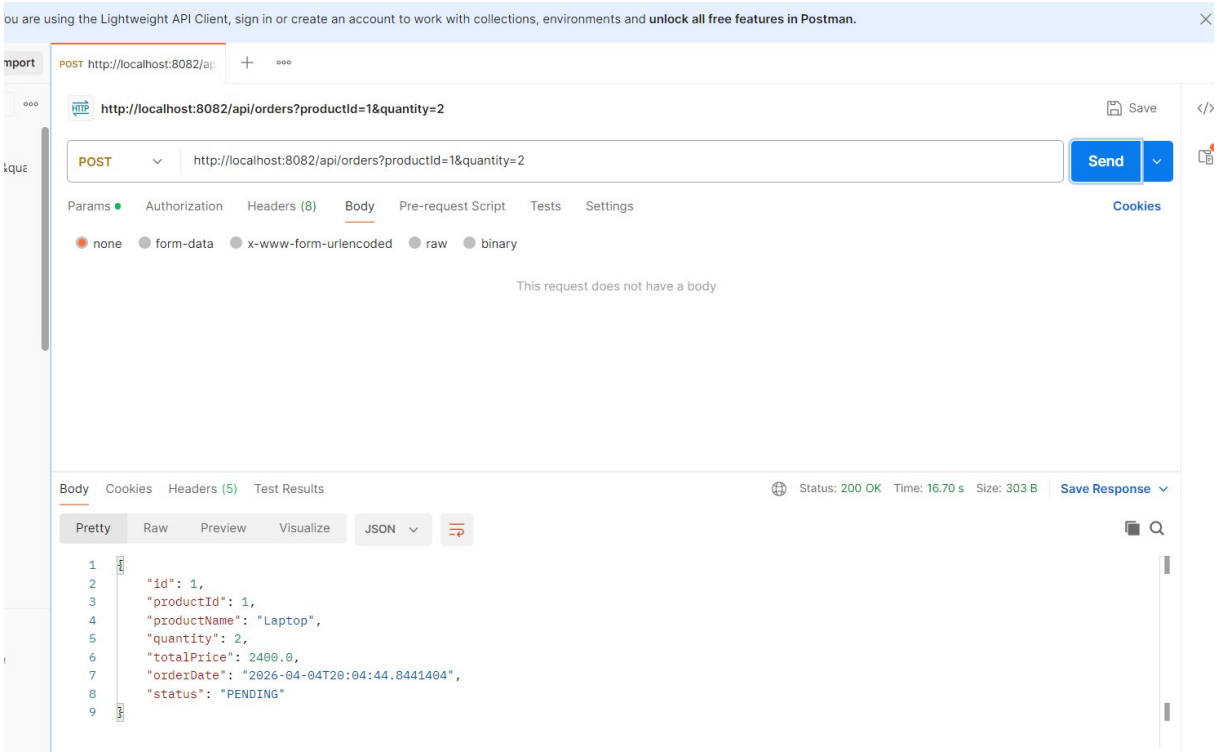

FIGURE 2 – POST /api/orders – Commande créée avec totalPrice=2400.0 et status=PENDING (200 OK)

6.4 Test 4 : Lister les commandes

Requête : `GET http://localhost:8082/api/orders`

La commande créée précédemment est bien persistée et retournée.

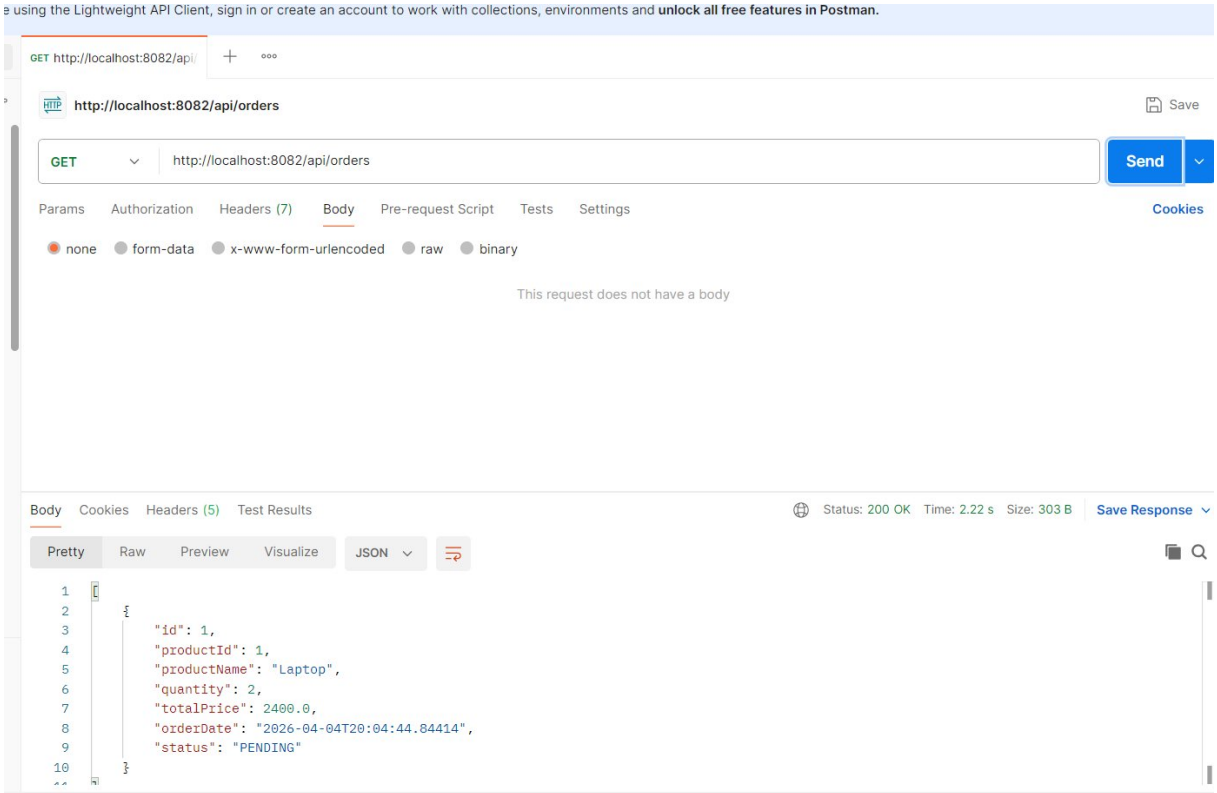

FIGURE 3 – GET /api/orders – Liste des commandes (200 OK)

7 Résultats et Analyse

7.1 Résumé des tests

Test	Endpoint	Méthode	Statut
Créer produit	/api/products	POST	200 OK ✓
Lister produits	/api/products	GET	200 OK ✓
Créer commande	/api/orders ?productId=1&quantity=2	POST	200 OK ✓
Lister commandes	/api/orders	GET	200 OK ✓

7.2 Points clés observés

— La communication inter-services via **OpenFeign** fonctionne correctement : Order Service récupère les informations du produit depuis Product Service.
— Le calcul du `totalPrice` est effectué automatiquement : $1200.00 \times 2 = 2400.0$.
— Le statut initial d'une commande est `PENDING`.
— Eureka Server assure la découverte automatique des services enregistrés.

8 Conclusion

Ce TP nous a permis de mettre en pratique les concepts fondamentaux de l'Architecture Orientée Services avec Spring Boot et Spring Cloud. Nous avons :

— Créé un serveur de découverte **Eureka** permettant l'enregistrement automatique des microservices.
— Développé un **Product Service** exposant une API REST CRUD complète.
— Développé un **Order Service** communiquant avec Product Service via **OpenFeign**, illustrant la communication inter-services.
— Testé l'ensemble de l'application avec **Postman** avec succès.

Cette architecture garantit l'indépendance, la scalabilité et la réutilisabilité de chaque service, conformément aux principes SOA.